

Introduction to R

Data Wrangling

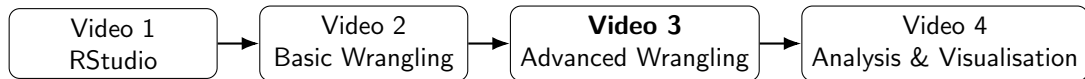
More Complex Wrangling, Merging and Data Reshaping

Isaac L. Singini

Biostatistics Research Unit (BRU)

South African Medical Research Council (SAMRC)

Introduction to R: Pre-Workshop Course



Today

Today we move from manipulating **one dataset** to combining, restructuring and preparing datasets for analysis.

By the end of this video, you should be able to:

- ① Combine multiple dplyr operations into a single workflow.
- ② Perform more complex variable manipulation.
- ③ Understand the difference between:
 - merging datasets;
 - appending datasets.
- ④ Use joins appropriately.
- ⑤ Convert data between wide and long formats.
- ⑥ Recognise why data structure matters for statistical analysis.

A Quick Recap: The Pipe

The pipe operator

`%>%`

or the modern R pipe

`|>`

means approximately:

`data |> operation()`

is read as:

“Take this data and pass it to the next operation.”

```
data |>
  filter(age >= 18) |>
  mutate(age_group = if_else(age >= 50,
                             "50+",
                             "<50")) |>
  arrange(age)
```

From Individual Commands to Workflows

Suppose we have:

```
patients
```

We want to:

- 1 keep adults;
- 2 create an age group;
- 3 keep selected variables;
- 4 sort by age.

Instead of four separate objects:

```
adults <- filter(patients, age  
  >= 18)  
adults2 <- mutate(adults, ...)  
adults3 <- select(adults2, ...)  
adults4 <- arrange(adults3, age  
  )
```

We can write the workflow as one sequence:

```
patients |>  
  filter(age >= 18) |>  
  mutate(...) |>  
  select(...) |>  
  arrange(age)
```

Real analyses rarely involve only one condition.

```
patients |>
  filter(
    age >= 18,
    sex == "Female",
    hiv_status == "Positive"
  )
```

Multiple conditions inside `filter()` are equivalent to:

$$A \wedge B \wedge C$$

That is:

Age $\geq$ 18 **AND** Female **AND** HIV positive.

You can also use:

- `&` – AND
- `|` – OR
- `!` – NOT

Creating Variables from Existing Variables

A common task is to derive analysis variables.

```
patients |>
  mutate(
    bmi = weight / height^2,
    age_group = case_when(
      age < 18 ~ "Child",
      age < 65 ~ "Adult",
      TRUE ~ "Older adult"
    )
  )
```

`case_when()` is especially useful when there are more than two categories.

Key idea

Derived variables should be created in a transparent and reproducible way rather than manually editing the dataset.

Conditional Variables: A More Realistic Example

Suppose we classify CD4 counts:

```
data |>
  mutate(
    cd4_group = case_when(
      cd4 < 200 ~ "<200",
      cd4 < 350 ~ "200--349",
      cd4 < 500 ~ "350--499",
      cd4 >= 500 ~ "500+",
      TRUE ~ NA_character_
    )
  )
```

The resulting variable is:

CD4	Category
< 200	< 200
200–349	200–349
350–499	350–499
≥500	500+

One of the most powerful patterns is:

`group_by()` + `mutate()`

For example, calculate the mean within each treatment group:

```
data |>
  group_by(treatment) |>
  mutate(
    mean_cd4 = mean(cd4, na.rm = TRUE)
  )
```

Unlike `summarise()`, `mutate()` retains the original observations.

`summarise()` → reduces rows

`mutate()` → usually preserves rows

Suppose we want to identify the highest value within each group.

```
data |>
  group_by(treatment) |>
  mutate(
    rank = min_rank(desc(cd4))
  )
```

We can then select:

```
data |>
  group_by(treatment) |>
  mutate(rank = min_rank(desc(cd4))) |>
  filter(rank <= 3)
```

This gives the top three observations within each treatment group.

Key idea

Grouping changes the meaning of subsequent operations.

When many variables need the same operation:

```
data |>
  summarise(
    across(
      c(age, weight, bmi),
      ~ mean(.x, na.rm = TRUE)
    )
  )
```

Or calculate missingness:

```
data |>
  summarise(
    across(
      everything(),
      ~ sum(is.na(.x))
    )
  )
```

This avoids repeatedly writing the same operation.

One operation → many variables

A Complete Wrangling Pipeline

A realistic workflow might look like:

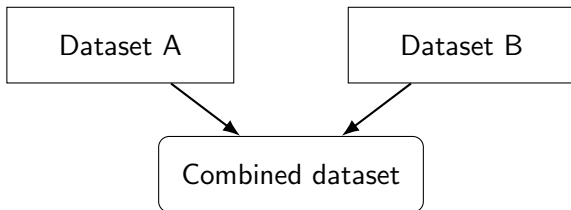
```
analysis_data <- patients |>
  filter(age >= 18, consent == 1) |>
  mutate(
    bmi = weight / height^2,
    age_group = case_when(
      age < 40 ~ "<40",
      age < 60 ~ "40--59",
      TRUE ~ "60+"
    )
  ) |>
  select(id, sex, age, age_group, treatment,
    bmi, outcome) |>
  arrange(treatment, age)
```

Key idea

Transparent code matters more than clever code.

Build a reproducible pipeline from raw data to analysis data.

These two concepts are often confused. **Merging / Joining**



Rows or variables are matched using one or more keys. **Appending**

Rows from Dataset A + Rows from Dataset B
⇓
More observations

Consider two datasets.

id	age	sex
101	34	F
102	51	M
103	29	F

+

id	treatment	outcome
101	A	0
102	B	1
103	A	0

The common variable is:

id

It is the **key** used to match observations.

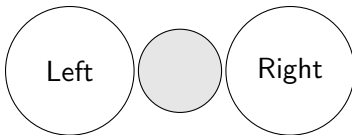
The most commonly useful join for analysis datasets is:

```
analysis <- patients |>
  left_join(
    laboratory,
    by = "id"
  )
```

Conceptually:

Keep everyone in the left dataset

and add matching information from the right dataset.



The Main Types of Joins

Function	Meaning
<code>left_join()</code>	Keep all rows from left
<code>right_join()</code>	Keep all rows from right
<code>inner_join()</code>	Keep matching rows only
<code>full_join()</code>	Keep all rows from both
<code>semi_join()</code>	Keep left rows with a match
<code>anti_join()</code>	Keep left rows without a match

For most analytic data assembly:

`left_join()`

is often the natural starting point.

Sometimes one variable is not enough. For example:

```
data1 |>
  left_join(
    data2,
    by = c(
      "id",
      "visit"
    )
  )
```

The match requires:

$$\begin{aligned} id_1 &= id_2 \\ \text{AND} \\ visit_1 &= visit_2 \end{aligned}$$

This is common in:

- longitudinal studies;
- repeated laboratory measurements;
- clinical visits;
- multi-level datasets.

A Critical Question Before Joining

Before running:

```
left_join(data1, data2, by = "id")
```

ask:

Is ID unique in each dataset?

Check:

```
data2 |>  
  count(id) |>  
  filter(n > 1)
```

If an ID appears multiple times:

one-to-many or many-to-many relationship

may occur. A join can therefore unexpectedly increase the number of rows.

Appending Data with `bind_rows()`

Suppose we have:

- participants recruited in 2025;
- participants recruited in 2026.

If they have the same variables:

```
combined <- bind_rows(  
  data_2025,  
  data_2026  
)
```

Conceptually:

$$\frac{\text{2025 participants}}{\text{2026 participants}}$$


One dataset with more rows

Appending Requires Compatible Variables

Suppose:

```
data_2025
```

contains:

id, age, sex, treatment

and:

```
data_2026
```

contains:

id, age, sex, treatment

Then:

```
bind_rows(data_2025, data_2026)
```

works naturally. But if variable names differ:

```
data_2025: treatment  
data_2026: trt
```

we should standardise them first.

```
data_2026 |>  
  rename(treatment = trt)
```

Consider repeated measurements. **Wide format**

id	cd4_baseline	cd4_month6	cd4_month12
101	180	260	390
102	240	310	420
103	150	230	350

The same concept is represented by multiple columns. **Long format**

id	visit	cd4
101	baseline	180
101	month6	260
101	month12	390
102	baseline	240

Wide → Long: `pivot_longer()`

Use:

```
pivot_longer()
```

Example:

```
long_data <- wide_data |>
  pivot_longer(
    cols = starts_with("cd4_"),
    names_to = "visit",
    values_to = "cd4"
  )
```

The transformation is:

Many measurement columns → One measurement column

and:

Column names → A variable

Long → Wide: pivot_wider()

The reverse operation is:

```
pivot_wider()
```

Example:

```
wide_data <- long_data |>
  pivot_wider(
    names_from = visit,
    values_from = cd4
  )
```

Conceptually:

Long $\xrightarrow{\text{pivot_wider()}}$ Wide

The choice depends on:

- the analysis;
- the software;
- the desired output;
- the structure of the original data.

Why Long Format Is Often Useful

Long data naturally represents:

Participant $\times$ Visit

For example:



This structure is particularly useful for:

- longitudinal data;
- repeated measures;
- mixed models;
- trajectory plots;
- grouped summaries.

Raw data: repeated CD4 measurements

```
id age cd4_baseline cd4_month6 cd4_month12
101 45 320 410 465
102 52 180 225 310
```

Reshape to analysis-ready long format

```
analysis_long <- raw_data |>
pivot_longer(
  starts_with("cd4_"),
  names_to = "visit",
  values_to = "cd4"
) |>
arrange(id, visit)
```

Key idea

Reshaping is one step in a larger wrangling pipeline.

Raw data: repeated CD4 measurements

id	age	cd4_baseline	cd4_month6	cd4_month12
101	45	320	410	465
102	52	180	225	310

Reshape to analysis-ready long format

```
analysis_long <- raw_data |>
  pivot_longer(
    starts_with("cd4_"),
    names_to = "visit",
    values_to = "cd4"
  ) |>
  arrange(id, visit)
```

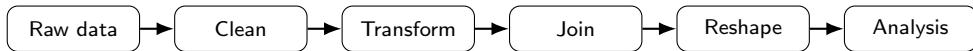
Key idea

Reshaping is one step in a larger wrangling pipeline.

Putting Everything Together

A realistic workflow may involve:

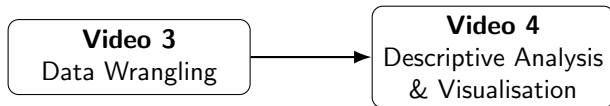
- 1 Import raw data.
- 2 Clean variable names.
- 3 Filter eligible observations.
- 4 Create derived variables.
- 5 Reshape repeated measurements.
- 6 Join demographic information.
- 7 Append datasets from different periods.
- 8 Validate the resulting dataset.



- ① Use pipes to build readable data-wrangling workflows.
- ② Use `mutate()` and `case_when()` for derived variables.
- ③ Distinguish carefully between:
 - joining datasets;
 - appending datasets.
- ④ Always understand your join key before merging.
- ⑤ Use `pivot_longer()` and `pivot_wider()` to reshape data.
- ⑥ Check the structure of the resulting dataset after every major transformation.

Good analysis begins with well-structured data.

Next: Descriptive Analysis and Visualisation



**From data preparation to statistical
communication**